

Relatório: T3 IA - Extrator Estruturado com LLM, RAG e Validação

Brunna Moura, Carlos Cruz, Rafael Queiroz, Victor Bitencourt

Definição do Problema

O grupo 4 desenvolveu uma aplicação para o projeto 2 da especificação: um extrator estruturado com LLM e validação. A aplicação recebe documentos em PDF ou texto bruto, extrai informações relevantes em campos definidos e valida a saída antes de devolvê-la ao usuário.

O trabalho também incorpora RAG para apoiar a extração. O sistema ingere uma base documental, segmenta o texto em chunks, gera embeddings, armazena os vetores em um banco persistente e recupera trechos relacionados ao documento analisado. Esses trechos ajudam o modelo a normalizar, classificar e checar informações, sem trocar a extração estruturada por uma resposta livre.

Base Documental

O trabalho usa dois conjuntos documentais com funções distintas. A **base documental do RAG** é o relatório da CPMI dos atos de 8 de janeiro de 2023. O **benchmark de avaliação** contém 30 textos curtos anotados. A tabela resume essa separação.

Conjunto	Conteúdo	Função	Ingerido no pgvector?
Base vetorial do RAG	PDF da CPMI, com 1.333 páginas	Fornecer chunks externos quando <code>top_k > 0</code>	Sim, 1.533 chunks
Benchmark anotado	30 textos de domínios administrativos variados	Servir como entrada da extração e referência para calcular as métricas	Não

O PDF da CPMI possui linguagem institucional, registros de depoimentos e menções a órgãos públicos, pessoas, datas e fatos distribuídos por várias seções. O volume atende ao requisito de usar uma base externa de conhecimento que justifique ingestão, vetorização e recuperação semântica. No fluxo de consulta RAG, o sistema localiza trechos desse relatório e cita as páginas de origem.

Os 30 textos anotados não são páginas nem recortes do relatório da CPMI. O script envia cada texto diretamente para `POST /api/extract`, sem armazená-lo no pgvector. Cada caso informa os valores esperados nos campos estruturados e o status de validação esperado. O protocolo usa esses casos para medir a extração, enquanto a CPMI fornece o contexto externo recuperado. Como os dois conjuntos tratam de domínios diferentes, a comparação também mede o efeito de contexto fora do domínio.

Pipeline de Ingestão

A rota `POST /api/ingest` executa a ingestão documental. O usuário envia um PDF e seleciona o embedder. A aplicação grava o arquivo em diretório temporário, extrai texto página por página com `PyMuPDF`, segmenta o conteúdo com `RecursiveCharacterTextSplitter`, gera embeddings para os chunks e salva o resultado no PostgreSQL com pgvector.

A numeração registrada começa em 1, como nas páginas exibidas ao usuário. A ingestão calcula embeddings e grava os chunks em lotes. A opção `reset_collection` remove os vetores do embedder selecionado antes da nova ingestão; o protocolo experimental usa essa opção para impedir duplicatas e garantir que as versões consultem o mesmo corpus.

A configuração padrão usa chunks de 2000 caracteres e overlap de 200 caracteres. O grupo escolheu esse tamanho após inspeção manual do documento da CPMI. Trechos curtos demais separavam atores, fatos e justificativas que apareciam no mesmo contexto discursivo. O overlap reduz perdas nas fronteiras entre chunks e mantém parte da continuidade textual usada na recuperação.

Arquitetura da Solução

O projeto usa Python 3.12, FastAPI e uma organização em portas e adaptadores. A camada `src/core/` concentra modelos de domínio, prompts, interfaces e o serviço de orquestração. A camada `src/infra/` implementa os adaptadores de banco, embeddings e LLM. A camada `src/api/` expõe as rotas HTTP, schemas e a interface web.

O `RAGService` concentra as operações principais:

- `embed_and_store`: recebe chunks, calcula embeddings e grava os vetores no banco.
- `query`: vetoriza a pergunta, recupera trechos similares e pede uma resposta ao LLM.
- `extract`: recebe o documento, recupera contexto opcional e pede um JSON estruturado ao LLM.

O armazenamento vetorial usa PostgreSQL com pgvector, acessado por SQLAlchemy. A aplicação define dois models ORM em `src/infra/models.py`: `ChunkModel`, usado pela tabela `chunk` com embeddings OpenAI-compatíveis de 768 dimensões, e `ChunkSpacyModel`, usado pela tabela `chunk_spacy` com embeddings spaCy de 300 dimensões. Cada model registra o texto do chunk, a página de origem e o vetor associado.

As migrations do Alembic criam as tabelas correspondentes no banco. Essa separação evita misturar vetores de dimensionalidades diferentes e permite comparar o comportamento do sistema com embeddings OpenAI-compatíveis e com embeddings locais do spaCy.

Recuperação Vetorial

A rota `POST /api/query` implementa o fluxo de pergunta e resposta fundamentada. O sistema transforma a pergunta em vetor, consulta o pgvector por similaridade e inclui os trechos recuperados no prompt. O LLM responde em português e informa as páginas usadas como fonte.

A rota `POST /api/extract` também usa recuperação vetorial quando `top_k` é maior que zero. Nesse caso, o sistema usa o texto do documento recebido como base para buscar chunks similares já ingeridos. A extração principal vem do documento enviado pelo usuário, enquanto os chunks recuperados funcionam como apoio para normalização, classificação e checagem.

Extração Estruturada e Validação

O fluxo principal do projeto 2 está em `POST /api/extract`. A rota aceita PDF, texto bruto ou ambos. Quando recebe PDF, a aplicação extrai o texto com `PDFLoader`. Para PDFs longos, o serviço seleciona até 24 blocos distribuídos entre o início, o meio e o fim, respeitando o limite de 24.000 caracteres. A consulta vetorial combina trechos do início, do meio e do fim em até 3.000 caracteres. Essa amostragem reduz o viés anterior, que considerava somente o começo do documento, mas ainda não equivale a uma leitura exhaustiva de todas as páginas.

O objeto de saída usa o seguinte formato:

```
{
  "document_type": null,
  "title": null,
  "main_event": null,
  "dates": [],
  "actors": [],
  "organizations": [],
  "facts": [],
  "evidence": [],
  "categories": [],
  "sources": [
    {"claim": "string", "page": 0}
  ],
  "confidence": "baixa"
}
```

A aplicação valida a resposta em camadas. Primeiro, tenta interpretar a resposta com `json.loads`. Se o modelo devolver texto fora do JSON, o serviço procura o objeto dentro da resposta. Se o JSON estiver malformatado, o sistema faz uma segunda chamada ao LLM para reparo. Depois disso, o serviço normaliza aliases em português, converte listas, ajusta fontes e valida o resultado com Pydantic.

Os campos obrigatórios são `document_type`, `title`, `main_event` e `facts`. A aplicação registra campos ausentes em `missing_required_fields` e classifica a saída como `valid`, `partial` ou `invalid`. O serviço valida a resposta do LLM antes de aplicar regras determinísticas. As regras ainda podem preencher campos óbvios para exibição, mas registram cada preenchimento em `inferred_fields`. Campos inferidos não transformam uma saída incompleta em `valid`.

Interface Web

A aplicação oferece uma interface web em `http://localhost:8000/`, implementada com HTML, CSS e JavaScript em `src/api/static/`. A interface organiza o fluxo de demonstração do projeto 2 em uma ação principal: **Ingerir e extrair PDF**.

Esse botão envia o mesmo arquivo para `POST /api/ingest` e, em seguida, para `POST /api/extract`. O usuário seleciona o PDF uma vez, escolhe o embedder e o valor de `top_k`, e a tela mostra o resultado estruturado com status de validação, confiança, campos extraídos, fontes e quantidade de chunks usados como contexto.

A tela também mantém a ação **Extrair sem ingestão**, usada para texto bruto ou para testar um documento sem atualizar a base vetorial. A área secundária de consulta RAG permite fazer perguntas sobre os chunks já armazenados.

Modelos e Componentes Utilizados

O projeto usa o SDK `openai` como cliente OpenAI-compatível. No ambiente Docker, o LLM padrão roda no Ollama, com `LLM_BASE_URL=http://ollama:11434/v1` e modelo `phi4-mini:latest`. Essa escolha permite executar o projeto localmente e ainda preserva a possibilidade de trocar o provedor por OpenAI, Gemini, Groq ou outro serviço compatível.

Para embeddings, a configuração padrão usa `nomic-embed-text:latest` no Ollama, com vetores de 768 dimensões. O projeto também possui um embedder com `pt_core_news_lg` do spaCy, com vetores de 300 dimensões, usado para comparação. O adaptador baixa o modelo spaCy no primeiro uso e mantém uma única instância em memória por processo da API.

As principais bibliotecas são FastAPI, SQLAlchemy assíncrono, asyncpg, pgvector, PyMuPDF, langchain-text-splitters, OpenAI SDK, Pydantic, Alembic e pytest. O ambiente completo roda com Docker Compose, usando serviços separados para API, PostgreSQL e Ollama.

Protocolo Experimental

O experimento responde à seguinte pergunta: adicionar cinco chunks recuperados da CPMI altera a qualidade da extração estruturada dos 30 textos anotados? Ele avalia a saída do extrator, não a correção factual do relatório da CPMI nem a qualidade de respostas do endpoint de perguntas.

O benchmark em `extraction_test_cases.json` contém 6 casos fáceis, 7 médios, 6 ambíguos, 6 com informação insuficiente e 5 que testam limites. Cada caso registra o texto de entrada, a categoria, os valores esperados por campo e o status de validação esperado. O script `run_extraction_evaluation.py` executa o protocolo em quatro etapas:

1. limpa a coleção spaCy e ingere o PDF da CPMI com `chunk_size=2000` e overlap de 200 caracteres;
2. envia cada um dos 30 textos para `POST /api/extract` com `top_k=0`, de modo que o LLM receba apenas o texto avaliado;
3. envia os mesmos 30 textos com `top_k=5`, de modo que o LLM receba o texto avaliado e cinco chunks recuperados da CPMI;
4. compara os campos e o status de cada resposta com as anotações do respectivo caso.

Cada texto gera duas respostas independentes, uma por configuração. O total é de 60 execuções: 30 sem recuperação e 30 com recuperação. As duas configurações usam o mesmo `phi4-mini:latest`, embedder spaCy, prompt, temperatura 0,1 e base vetorial com 1.533 chunks. O uso do contexto recuperado é a única variável da comparação.

O avaliador calcula as métricas sobre cada resposta:

- **JSON/schema válido:** a resposta segue os tipos e a estrutura esperados; essa métrica não verifica se o conteúdo está correto;
- **campos obrigatórios completos:** `document_type`, `title`, `main_event` e `facts` estão preenchidos e não dependem de valores inferidos por fallback;
- **status correto:** `valid`, `partial` ou `invalid` corresponde ao status anotado; nos casos esperados como `partial`, uma resposta `invalid` também indica que o sistema não aceitou informação insuficiente como válida;
- **falso `valid`:** um dos seis casos insuficientes recebeu incorretamente o status `valid`;
- **cobertura anotada:** proporção dos 206 valores esperados que aparecem nos campos retornados, após normalização de caixa, acentos e pontuação;
- **latência e chunks usados:** tempo da requisição e quantidade de trechos recuperados da CPML.

A cobertura mede recall sobre os valores anotados. Ela não calcula precisão e, portanto, não penaliza todos os valores adicionais ou incorretos produzidos pelo modelo. O grupo precisa complementar essa métrica com inspeção humana das saídas. O JSON completo de cada execução fica em `docs/extraction_performance_results.json`.

Resultados

Os resultados medem a extração estruturada dos 30 textos do benchmark. A coluna `top_k=0` mostra o desempenho quando o modelo recebeu apenas cada texto avaliado. A coluna `top_k=5` mostra o desempenho nos mesmos textos quando o modelo também recebeu cinco chunks da CPML. Os números não representam perguntas respondidas sobre a CPML.

As 60 execuções terminaram sem erro de transporte ou da API. A tabela apresenta as contagens e os percentuais agregados de cada configuração.

Métrica	<code>top_k=0</code>	<code>top_k=5</code>
Execuções concluídas	30	30
JSON/schema válidos	30/30 (100,0%)	30/30 (100,0%)
Campos obrigatórios completos	18/30 (60,0%)	14/30 (46,7%)
Status de validação correto	24/30 (80,0%)	20/30 (66,7%)
Falsos <code>valid</code> em casos insuficientes	0/6 (0,0%)	0/6 (0,0%)
Valores anotados recuperados	148/206 (71,8%)	112/206 (54,4%)
Latência média	18,86 s	68,78 s
Chunks de contexto médios	0,0	5,0

O resultado de 100% em JSON/schema significa que as 60 respostas puderam ser interpretadas e validadas pela aplicação. Ele não significa que todos os campos estavam completos ou corretos. Sem RAG, 18 das 30 respostas preencheram os quatro campos obrigatórios sem depender de fallback; com cinco chunks da CPML, esse total caiu para 14. O status coincidiu com o esperado em 24 casos sem RAG e em 20 casos com RAG. Nenhuma configuração classificou como **valid** os seis textos que tinham informação insuficiente.

As anotações contêm 206 valores esperados distribuídos pelos campos dos 30 casos. A configuração sem RAG recuperou 148 desses valores; a configuração com RAG recuperou 112. A diferença de 36 valores corresponde à queda de 17,4 pontos percentuais na cobertura. A latência média subiu de 18,86 para 68,78 segundos, aproximadamente 3,6 vezes.

Categoria	Cobertura k=0	Cobertura k=5	Status k=0	Status k=5
Fácil	80,4%	64,7%	50,0%	50,0%
Médio	73,4%	51,6%	85,7%	57,1%
Ambíguo	68,0%	54,0%	66,7%	66,7%
Base insuficiente	100,0%	100,0%	100,0%	100,0%
Limite	62,5%	45,0%	100,0%	60,0%

Na categoria de informação insuficiente, a cobertura de 100% corresponde a apenas um valor anotado, recuperado nas duas configurações. A métrica relevante para esses seis casos é a ausência de falsos **valid**, não a cobertura.

Os resultados sustentam uma conclusão restrita ao protocolo executado: adicionar contexto da CPML a textos de outros domínios introduziu ruído, reduziu a cobertura e aumentou o tempo de resposta. O experimento não demonstra que RAG prejudica a extração em geral. Para avaliar o benefício do RAG, o grupo ainda precisa repetir o protocolo com uma base externa do mesmo domínio dos textos avaliados ou com um limiar que rejeite chunks pouco similares. O relatório gerado automaticamente em [docs/extraction_performance_report.md](#) e o JSON completo permitem auditar cada caso.

Testes e Reprodutibilidade

O repositório inclui testes com pytest. Os testes unitários cobrem chunking, extração, validação, métricas do experimento e interface estática. Os testes de repositório e das rotas HTTP são testes de integração e requerem PostgreSQL, Ollama e a API em execução.

O grupo executou verificações direcionadas durante o desenvolvimento:

```
uv run pytest tests/test_extract.py tests/test_extraction_evaluation.py \
    tests/test_repository.py tests/test_static_ui.py -q
uv run ruff check src tests run_extraction_evaluation.py
python -m compileall src tests
```

O protocolo completo pode ser repetido com:

```
uv run python run_extraction_evaluation.py \  
  --prepare-corpus --embedder spacy --top-k 0,5
```

Esse comando recria a coleção, executa 60 extrações e gera os relatórios Markdown e JSON usados nesta seção.

Análise Crítica

O contexto RAG só ajuda a extração quando a base externa trata do mesmo domínio do documento recebido. Neste protocolo, os documentos curtos cobrem domínios administrativos variados, enquanto a base vetorial contém o relatório da CPMI. Essa escolha testa também o risco de contexto irrelevante. Uma implantação real deve separar coleções por domínio ou aplicar um limiar de similaridade antes de incluir chunks no prompt.

O embedder spaCy usa vetores estáticos e captura menos contexto semântico do que modelos baseados em Transformers. Ele foi mantido fixo nas duas versões porque o objetivo do experimento é isolar o efeito da recuperação, não comparar modelos de embedding. Uma comparação entre embedders exigiria repetir a ingestão do mesmo corpus e todas as 60 execuções sob as mesmas condições.

Na extração estruturada, a principal dificuldade está na disciplina do LLM em retornar JSON válido e completo. A validação com Pydantic e o reparo automático reduzem falhas operacionais. Os fallbacks melhoram a apresentação, mas agora preservam a ausência original em `missing_required_fields` e registram a origem em `inferred_fields`. Essa distinção evita usar uma heurística como evidência de que o LLM extraiu corretamente o campo.

A cobertura automática também tem limites. Ela confirma que os valores anotados aparecem na saída, mas não penaliza todas as entidades ou fatos adicionais. A equipe precisa revisar uma amostra das saídas completas antes da apresentação, sobretudo nos casos ambíguos, de prompt injection e de base insuficiente.

A interface web melhorou a demonstração do projeto. O fluxo com um único upload evita que o usuário ingira um PDF e depois precise enviá-lo de novo em outra área da tela. Essa decisão tornou a apresentação mais coerente com o projeto 2, pois a extração estruturada aparece como tarefa principal e a ingestão entra como etapa necessária para fornecer contexto RAG.

Considerações sobre Uso de IA Generativa

A equipe usou IA generativa como apoio em etapas pontuais. Agentes de IA ajudaram na criação de casos de teste, no apoio aos scripts de comparação de `top_k`, na geração do diagrama de arquitetura em DOT e na revisão de partes da documentação. O grupo revisou o código gerado, executou testes e ajustou o comportamento da aplicação. As anotações dos 30 casos ficam legíveis no JSON para que os integrantes confirmem cada valor esperado e defendam os critérios durante a apresentação.

Na etapa da interface, a discussão começou com a seguinte pergunta feita ao assistente:

baseado no contexto do AGENTS.md vc acha valido criar uma interface para esse projeto?

A partir dessa discussão, o grupo decidiu criar uma interface simples para a demonstração. Depois dos testes manuais, o grupo ajustou a tela para chamar `/api/extract` e consolidou o fluxo em **Ingerir e extrair PDF**, que executa `/api/ingest` antes de `/api/extract` com o mesmo arquivo.

Conclusão

O projeto entrega uma aplicação com LLM, base documental externa, ingestão, embeddings, armazenamento vetorial persistente, recuperação por similaridade, extração estruturada e validação. A arquitetura em portas e adaptadores facilita a troca de embedders e provedores de LLM, enquanto a interface web concentra o fluxo de demonstração em uma experiência mais direta.

O protocolo de 30 casos avalia diretamente a extração estruturada e compara a mesma solução com e sem recuperação vetorial. As anotações, o avaliador e as saídas completas permanecem no repositório, permitindo repetir os números e inspecionar falhas por campo. Neste experimento, o contexto fora do domínio reduziu cobertura, completude e acerto do status e aumentou a latência. A próxima etapa deve acrescentar uma métrica de precisão para valores não anotados e testar coleções externas do mesmo domínio de cada documento.

Referências

- Especificação do trabalho: [docs/T3_Especificacao_LLMs_RAG_Validacao.pdf](#)
- Casos anotados: [extraction_test_cases.json](#)
- Resultados completos: [docs/extraction_performance_results.json](#)
- Documentação do pgvector: <https://github.com/pgvector/pgvector>
- Modelos spaCy em português: <https://spacy.io/models/pt>
- Ollama: <https://ollama.com/>
- FastAPI: <https://fastapi.tiangolo.com/>